Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №1

Децентралізовані застосунки

Технології віддаленого виклику процедур – GRPC

Виконав
аспірант групи ІТ-41ф

Новиков Д. М.

Перевірив:

д.т.н. Гаврилко Євген Володимирович

Київ 2026

Мета роботи: Використовуючи технологію GRPC, розробити клієнт-серверну програму відповідно до індивідуального завдання.

Завдання роботи:

Використовуючи технологію GRPC, розробити клієнт-серверну програму відповідно до індивідуального завдання.

Варіант 4: Захищена (шифрування XOR) передача повідомлень по мережі.

Хід роботи:

Використані технології:

- Платформа: .NET 9 (C#).
- Мережева взаємодія: gRPC (поверх HTTP/2) та Protocol Buffers.
- Інфраструктура: Docker та Docker Compose.

Архітектура Системи:

Система побудована за мікросервісним принципом і складається з трьох основних компонентів:

- **gRPC Server** (Брокер повідомлень): Центральний вузол, який не має доступу ззовні. Відповідає за генерацію ключів, зберігання активних підключень (потоків) та безпечну маршрутизацію (перешифрування) повідомлень між клієнтами.
- **Консольний Клієнт**: Додаток для терміналу, що демонструє роботу багатопотоковості при прямому gRPC-підключенні до сервера.
- **API Client (Gateway)**: Веб-шлюз, який приймає класичні HTTP/REST запити через інтерфейс Swagger, транслює їх у бінарні gRPC запити та керує чергою вхідних повідомлень.

Безпека та Управління Сесіями:

Щоб уникнути вразливості статичних ключів, у системі реалізовано динамічний обмін ключами - Handshake: При старті будь-який клієнт ініціює з'єднання. Сервер генерує криптографічно стійкий масив байтів (Session Key) та унікальний ідентифікатор (Session ID), зберігаючи їх у потокобезпечному словнику ConcurrentDictionary.

Маршрутизація з перешифруванням: Коли Клієнт А пише Клієнту Б, повідомлення шифрується ключем Клієнта А. Сервер розшифровує його, знаходить сесію Клієнта Б, зашифровує текст його унікальним ключем і

лише потім відправляє. Це гарантує надійність навіть при перехопленні трафіку.

Моделі комунікації gRPC:

У контракті (.proto файлі) реалізовано дві різні парадигми взаємодії поверх одного мультиплексованого HTTP/2 з'єднання:

- Unary RPC (SendMessage): Класична модель "запит-відповідь". Використовується для прямого "пінгування" сервера (команда /server у консолі);
- Bidirectional Streaming RPC (ChatStream): Двонаправлений потік. З'єднання залишається постійно відкритим, що дозволяє клієнтам надсилати та отримувати повідомлення один від одного в режимі реального часу без необхідності постійно опитувати сервер (Long Polling).

```
syntax = "proto3";

option csharp_namespace = "GrpcSecure.Shared.Protos";

package securemessaging;

service SecureCommunication {
// 1. Initial connection to obtain a session and a key.
  rpc Handshake (HandshakeRequest) returns (HandshakeResponse);

  // 2. Unary call: Simple client-to-server encrypted message (Echo/Test).
  rpc SendMessage (SecureMessage) returns (SecureMessage);

  // 3. Bidirectional stream: Real-time client-to-client chat relay.
  rpc ChatStream (stream ChatMessage) returns (stream ChatMessage);
}

// --- Messages for Handshake ---
message HandshakeRequest {
}

message HandshakeResponse {
  string session_id = 1;
  bytes xor_key = 2; // The server provides a dynamically generated key.
}

// --- Messages for Unary SendMessage ---
message SecureMessage {
  bytes payload = 1; // Encrypted data.
}

// --- Messages for Bidirectional ChatStream ---
message ChatMessage {
  // The ID of the client this message is intended for.
  string target_session_id = 1;

  // The ID of the client who sent this message (populated by the server).
  string sender_session_id = 2;

  // The encrypted data.
  bytes payload = 3;
}
```

Реалізація клієнтських додатків:

Обидва типи клієнтів адаптовані для роботи з асинхронними потоками даних:

- **Консольний клієнт:** Розділений на два паралельні процеси. Фонове завдання (Background Listener) безперервно читає вхідний gRPC-потік, розшифровує повідомлення та виводить їх на екран. Головний потік обробляє ввід користувача;
- **API Клієнт (Gateway):** Оскільки REST не підтримує постійні з'єднання, шлюз інкапсулює gRPC-потік усередині Singleton-сервісу SecureChatSession. Усі вхідні повідомлення з потоку складаються у локальну потокобезпечну чергу (ConcurrentQueue). REST-ендпоінти (/chat/send, /chat/inbox) працюють виключно з цією пам'яттю, виступаючи мостом між синхронним веб-світом та асинхронним gRPC-чатом.

Інфраструктура та Розгортання (Docker):

Використання Docker Compose дозволяє розгорнути сервер та кілька незалежних API-клієнтів однією командою.

Усі сервіси знаходяться в ізольованій віртуальній мережі (bridge network). gRPC Сервер прихований від зовнішнього світу, а спілкування контейнерів відбувається за DNS-іменами середовища.

Налаштування (наприклад, адреса сервера) передаються через Environment Variables, що робить систему готовою до масштабування.

Приклад використання:

Запустимо наш docker-compose сценарій, який містить 1 gRPC сервер (наш брокер повідомлень) та 2 API-клієнти (шлюзи). Також додатково запустимо консольний клієнт для взаємодії та демонстрації роботи системи:

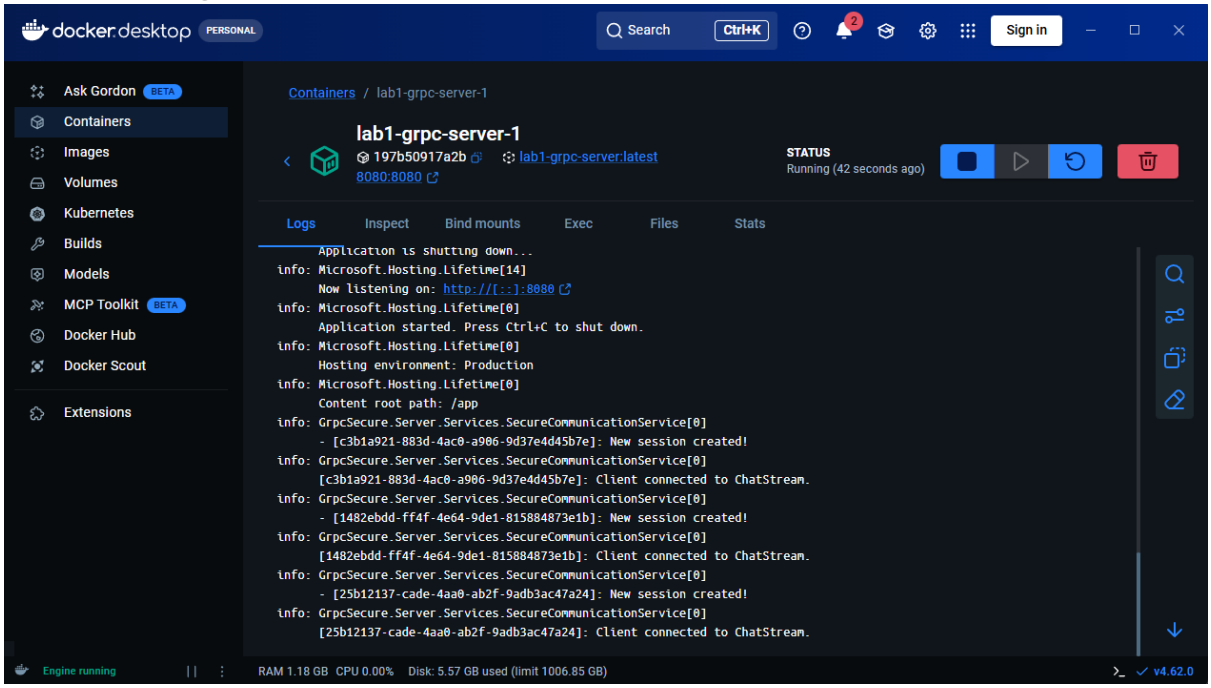
docker-compose:

```
services:
  # The central gRPC Server
  grpc-server:
    build:
      context: .
      dockerfile: GrpcSecure.Server/Dockerfile
    ports:
      - "8080:8080"
    networks:
      - secure-net
    restart: unless-stopped

  # API Client 1 - Exposed on Host Port 8081
  api-client-1:
    build:
```

```yaml
      context: .
      dockerfile: GrpcSecure.ApiClient/Dockerfile
    environment:
      - GrpcServer__Endpoint=http://grpc-server:8080
      - ASPNETCORE_URLS=http://+:80
    ports:
      - "8081:80"
    depends_on:
      - grpc-server
    networks:
      - secure-net
    restart: unless-stopped

  # API Client 2 - Exposed on Host Port 8082
  api-client-2:
    build:
      context: .
      dockerfile: GrpcSecure.ApiClient/Dockerfile
    environment:
      - GrpcServer__Endpoint=http://grpc-server:8080
      - ASPNETCORE_URLS=http://+:80
    ports:
      - "8082:80"
    depends_on:
      - grpc-server
    networks:
      - secure-net
    restart: unless-stopped

# Define the isolated virtual network
networks:
  secure-net:
    driver: bridge
```

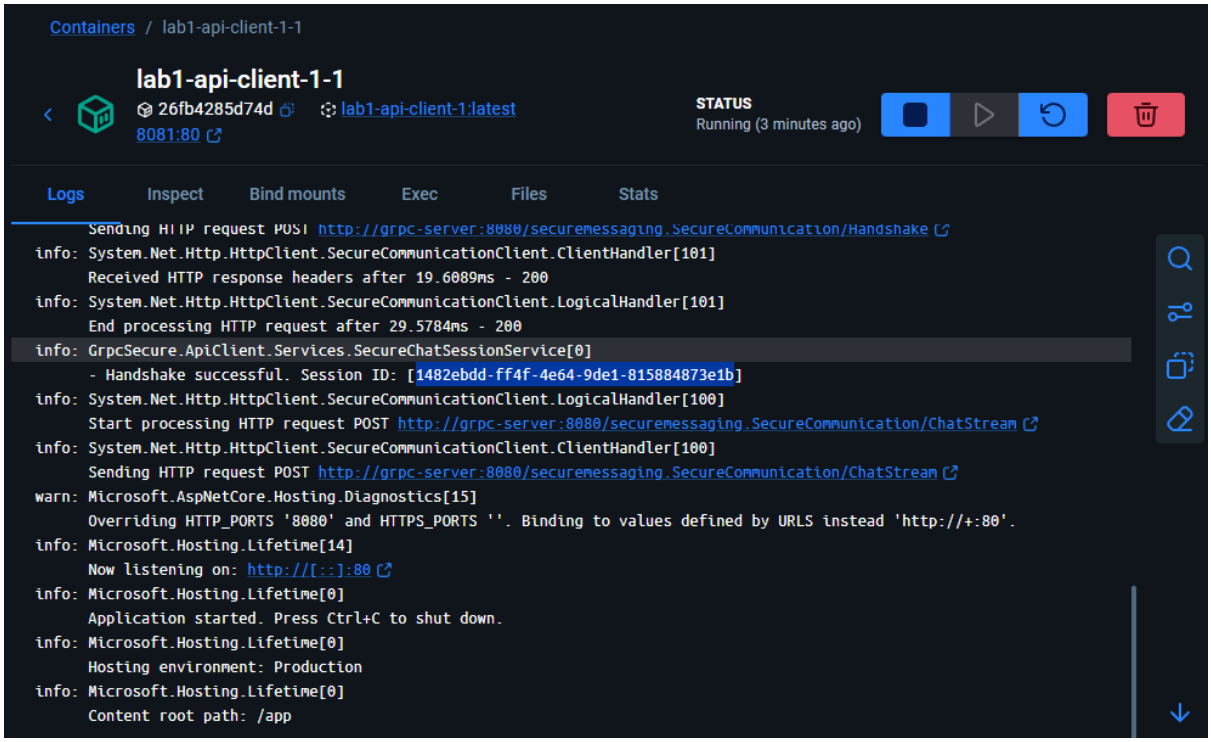
Перевіримо логи наших компонентів під час запуску:

- gRPC Server:



Як бачимо з логів сервера, успішно створено 3 незалежні сесії (дві для API-клієнтів та одна для консольного клієнта). Для кожного підключення згенеровано унікальний ідентифікатор та відкрито двонаправлений потік ChatStream;

- APIClient1:



Шлюз успішно виконав процес Handshake із gRPC сервером, отримав свій унікальний ключ для XOR-шифрування та сесійний ідентифікатор [1482ebdd-ff4f-4e64-9de1-815884873e1b]. Фоновий потік для прослуховування повідомлень запущено;

- APIClient2:

```
Containers / lab1-api-client-2-1
lab1-api-client-2-1
e90088a8f8d1   lab1-api-client-2:latest
8082:80
STATUS
Running (4 minutes ago)
Logs   Inspect   Bind mounts   Exec   Files   Stats
      Sending HTTP request POST http://grpc-server:8080/securemessaging.SecureCommunication/Handshake
info: System.Net.Http.HttpClient.SecureCommunicationClient.ClientHandler[101]
      Received HTTP response headers after 64.4574ms - 200
info: System.Net.Http.HttpClient.SecureCommunicationClient.LogicalHandler[101]
      End processing HTTP request after 74.9886ms - 200
info: GrpcSecure.ApiClient.Services.SecureChatSessionService[0]
      - Handshake successful. Session ID: [c3b1a921-883d-4ac0-a906-9d37e4d45b7e]
info: System.Net.Http.HttpClient.SecureCommunicationClient.LogicalHandler[100]
      Start processing HTTP request POST http://grpc-server:8080/securemessaging.SecureCommunication/ChatStream
info: System.Net.Http.HttpClient.SecureCommunicationClient.ClientHandler[100]
      Sending HTTP request POST http://grpc-server:8080/securemessaging.SecureCommunication/ChatStream
warn: Microsoft.AspNetCore.Hosting.Diagnostics[15]
      Overriding HTTP_PORTS '8080' and HTTPS_PORTS ''. Binding to values defined by URLS instead 'http://+:80'.
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://[::]:80
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /app
```

Аналогічно до першого клієнта, другий шлюз підключився та отримав свій ідентифікатор [c3b1a921-883d-4ac0-a906-9d37e4d45b7e];

- Console Client:

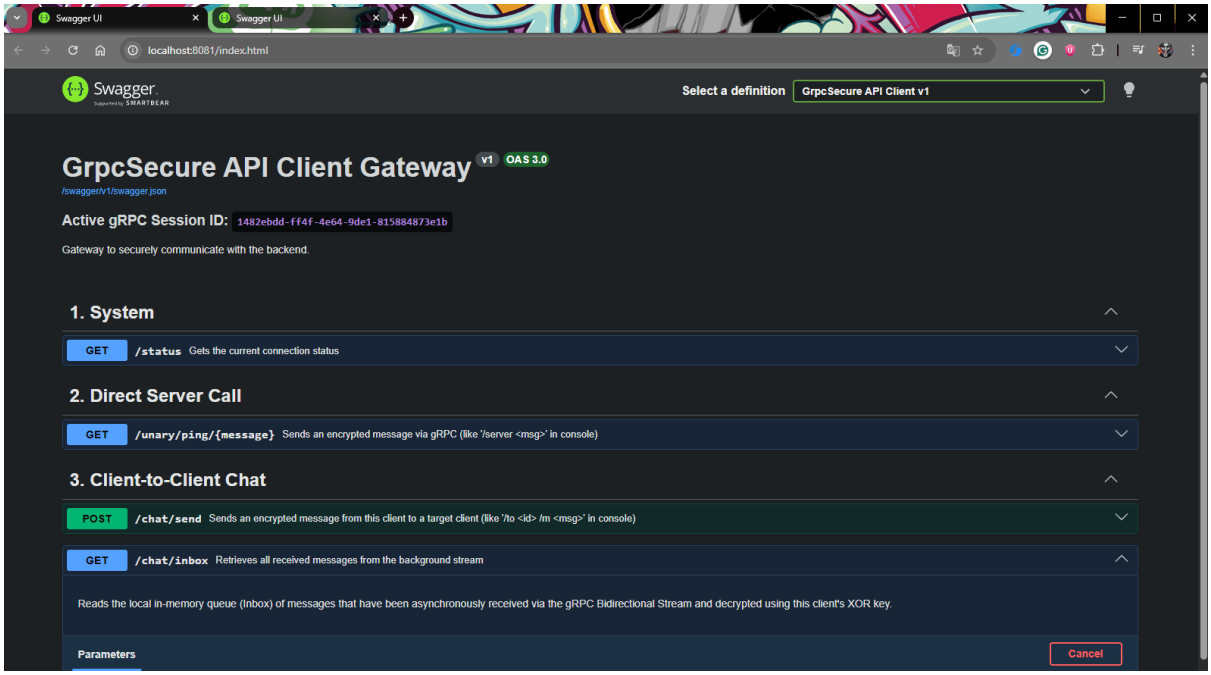
```
V:\Winda_BB_2025\SSD_backu
=== Secure gRPC Client ===
info: SecureChatAppV2[0]
      - Initiating handshake...
info: System.Net.Http.HttpClient.SecureCommunicationClient.LogicalHandler[100]
      Start processing HTTP request POST http://localhost:8080/securemessaging.SecureCommunication/Handshake
info: System.Net.Http.HttpClient.SecureCommunicationClient.ClientHandler[100]
      Sending HTTP request POST http://localhost:8080/securemessaging.SecureCommunication/Handshake
info: System.Net.Http.HttpClient.SecureCommunicationClient.ClientHandler[101]
      Received HTTP response headers after 75.8873ms - 200
info: System.Net.Http.HttpClient.SecureCommunicationClient.LogicalHandler[101]
      End processing HTTP request after 86.441ms - 200
info: SecureChatAppV2[0]
      - Handshake successful. Session ID: [25b12137-cade-4aa0-ab2f-9adb3ac47a24].

==================================================
YOUR ID: 25b12137-cade-4aa0-ab2f-9adb3ac47a24
==================================================

info: System.Net.Http.HttpClient.SecureCommunicationClient.LogicalHandler[100]
      Start processing HTTP request POST http://localhost:8080/securemessaging.SecureCommunication/ChatStream
info: System.Net.Http.HttpClient.SecureCommunicationClient.ClientHandler[100]
      Sending HTTP request POST http://localhost:8080/securemessaging.SecureCommunication/ChatStream
[System]: Chat started!
[System]: 1. Type '/to <TargetSessionID>' to set a chat recipient. After that, you can enter messages to be sent to the
target recipient.
[System]: 2. Type '/to <TargetSessionID> /m <message>' to set recipient and send instantly.
[System]: 3. Type '/server <message>' to ping the server directly (Unary call).
[System]: 4. Type 'exit' to quit.
>
```

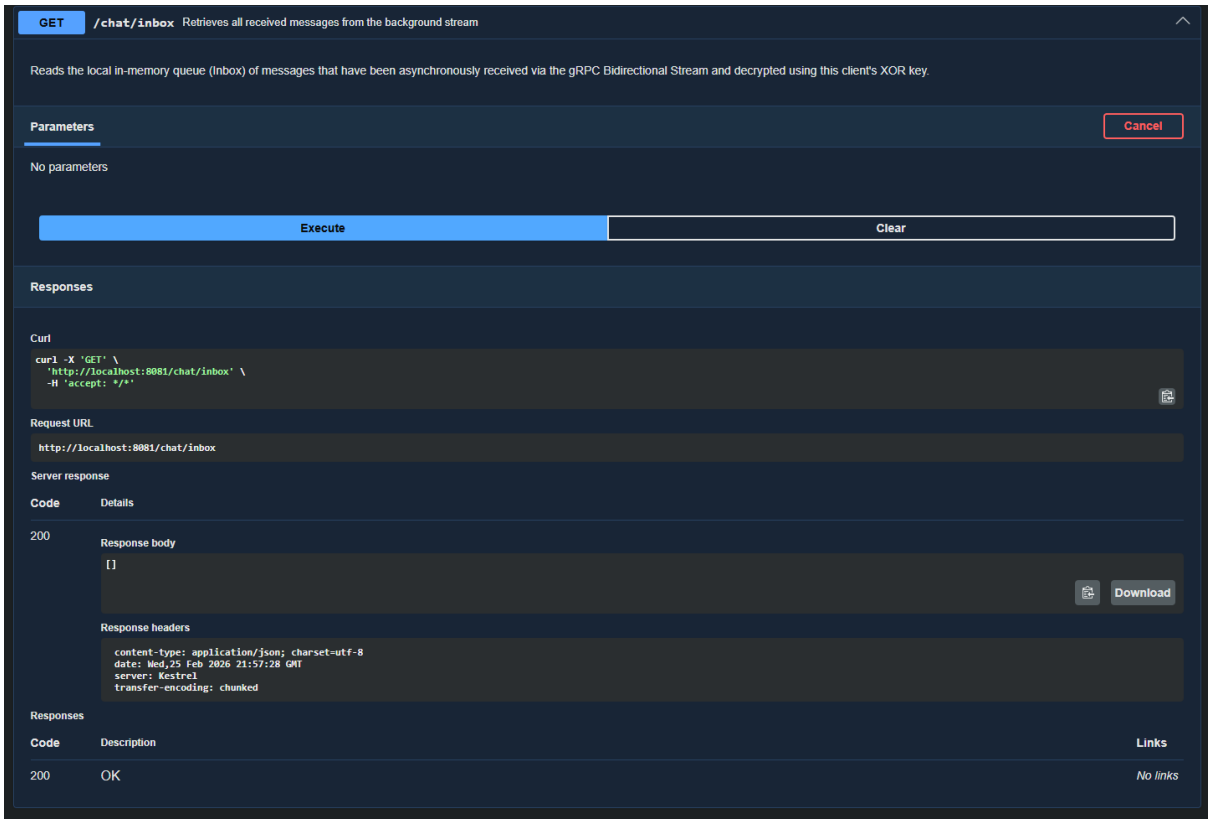
Консольний додаток також пройшов ініціалізацію, отримав ID [25b12137-cade-4aa0-ab2f-9adb3ac47a24] і вивів на екран меню команд.

Сценарій взаємодії (Обмін повідомленнями):

Крок 1: Відправка повідомлення з терміналу (Console -> APIClient1)

Відкриваємо браузер з інтерфейсом Swagger для API Client 1 (наприклад, http://localhost:8081). На боці API Client 1 перевіримо вхідні повідомлення та верифікуємо, що ще не було отримано жодного повідомлення:





Тіло відповіді порожнє, на клієнт ще не надходило жодного повідомлення.

У консольному клієнті використовуємо команду для миттєвої зміни адресата та відправки тексту. Як ціль вказуємо Session ID нашого першого API-клієнта:

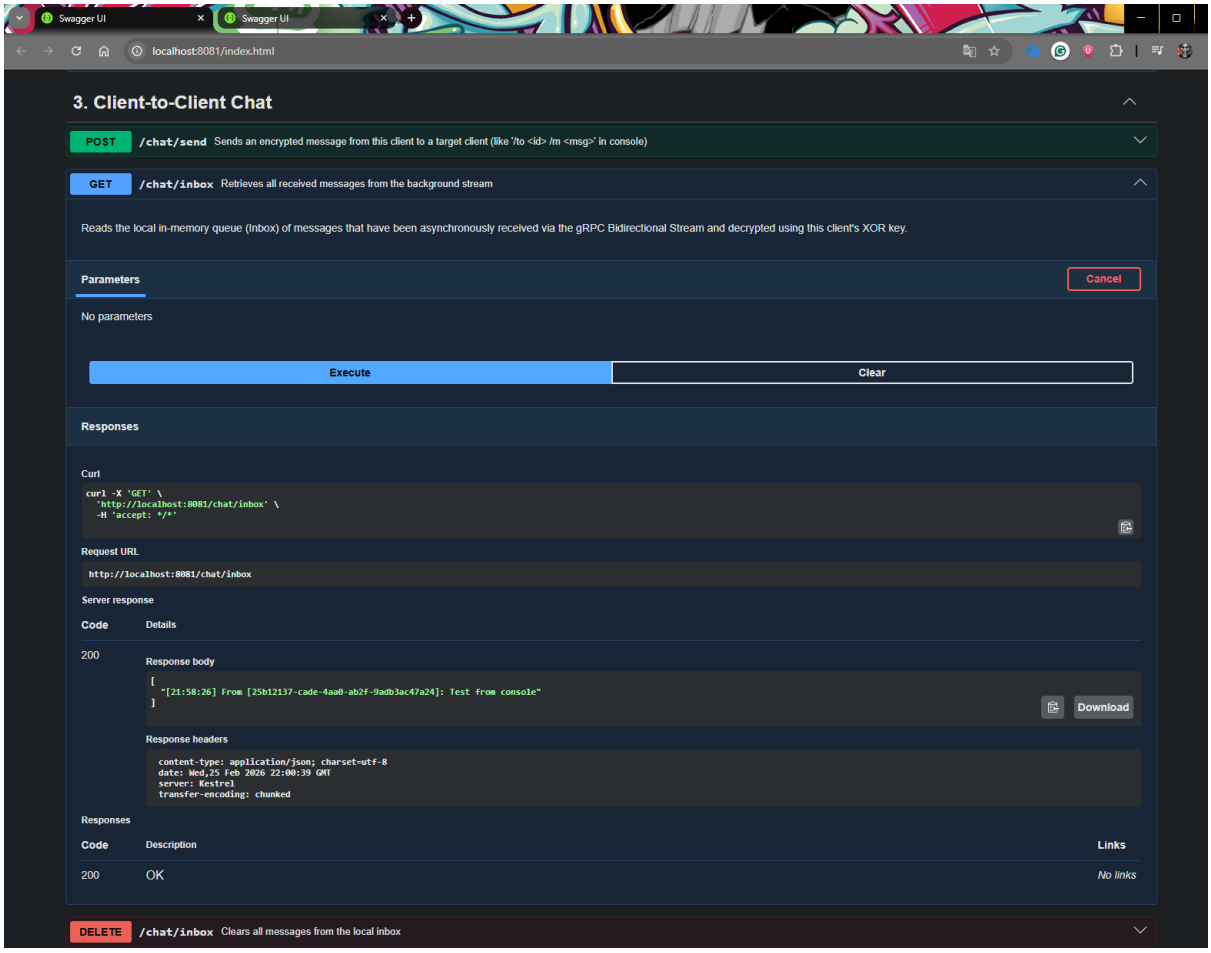
> /to 1482ebdd-ff4f-4e64-9de1-815884873e1b /m Test from console

```
[System]: 4. Type 'exit' to quit.
> /to 1482ebdd-ff4f-4e64-9de1-815884873e1b /m Test from console
[System]: Recipient set to [1482ebdd-ff4f-4e64-9de1-815884873e1b].
>
```

Програма автоматично зашифрує текст унікальним ключем консольного клієнта і відправить його у постійно відкритий потік (Bidirectional Stream). Сервер перехопить ці байти, розшифрує їх, знайде сесію API Client 1, зашифрує текст його унікальним ключем і перешле далі.

Крок 2: Отримання та відповідь через веб-інтерфейс (Swagger)

Повернемося до API Client 1, перевіримо вхідні повідомлення та верифікуємо, що повідомлення від консольного клієнта успішно доставлено:



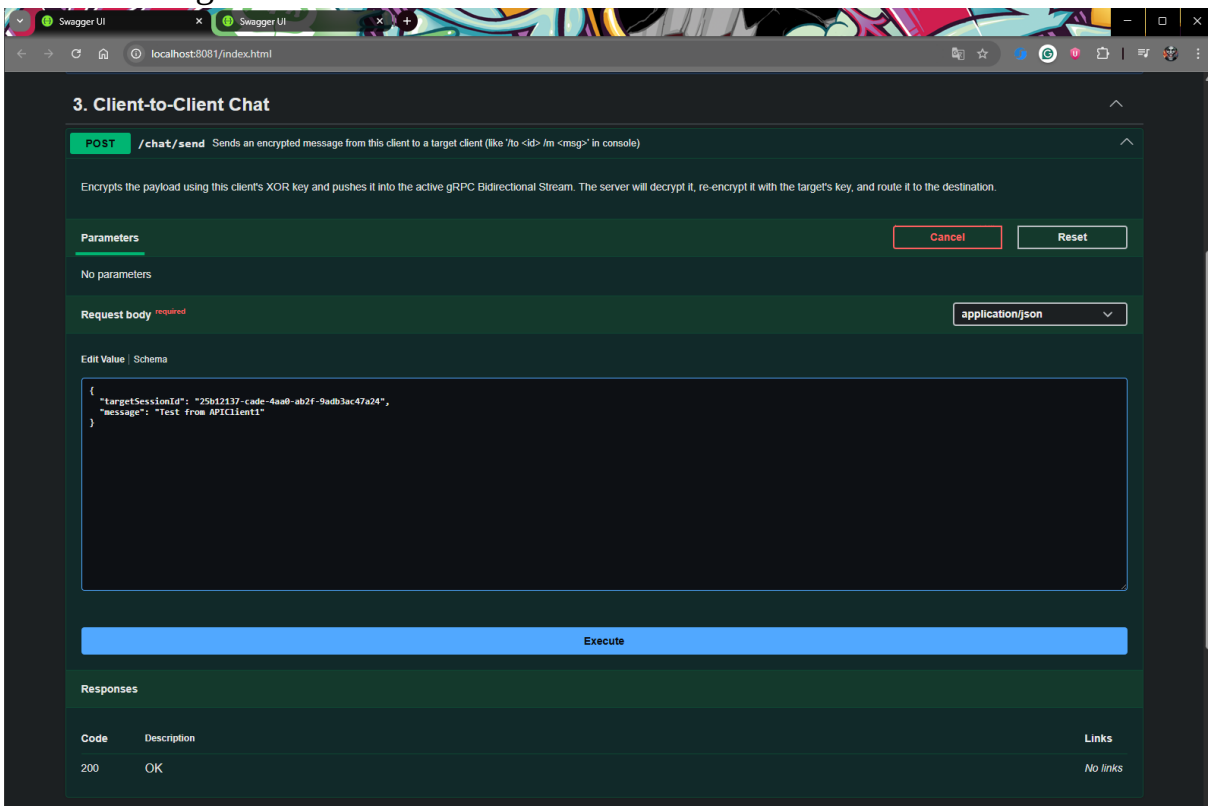
Щоб прочитати повідомлення, виконуємо GET-запит до ендпоінту /chat/inbox. У відповіді (Response body) ми побачимо наше розшифроване

повідомлення "Test from console" разом із міткою часу, яке фоновий потік шлюзу вже встиг обробити.

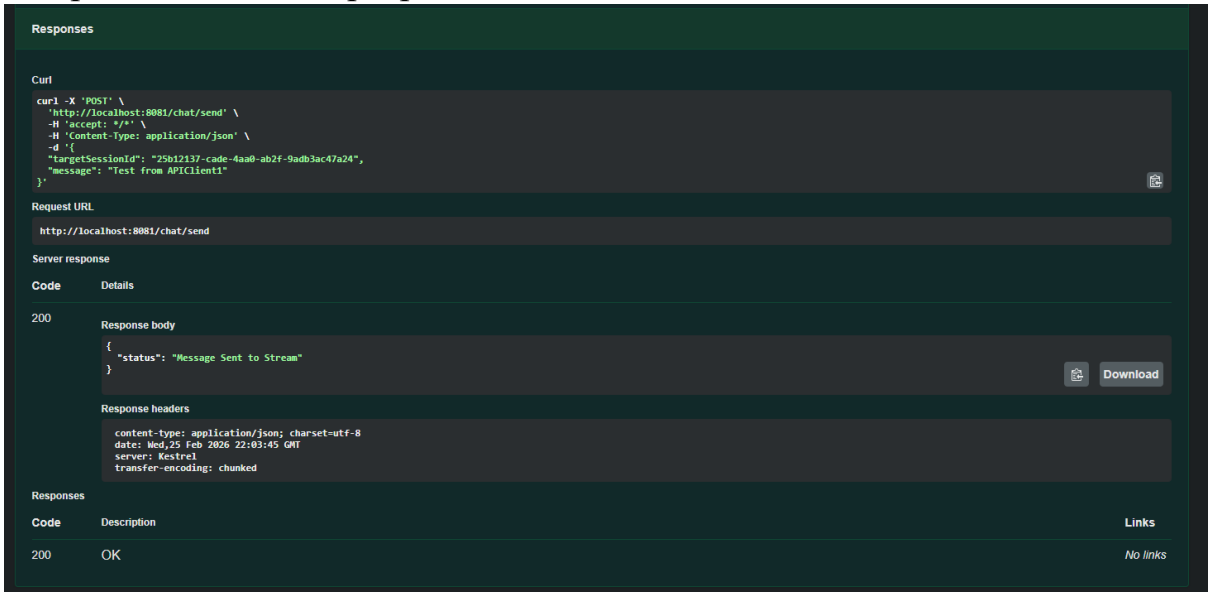
Щоб відповісти, переходимо до POST-запиту /chat/send. У тілі запиту (Request body) вказуємо:

TargetSessionId: 25b12137-cade-4aa0-ab2f-9adb3ac47a24 (ID консолі)

Message: "Test from APIClient1"



Натискаємо Execute. API-шлюз зашифрує це повідомлення і відправить його на сервер.



Крок 3: Асинхронне отримання в консолі:

Оскільки консольний клієнт працює у два потоки, фоновий слухач (Background Listener) миттєво отримає цю відповідь і виведе її на екран: [1482ebdd-ff4f-4e64-9de1-815884873e1b] says: Test from APIClient1

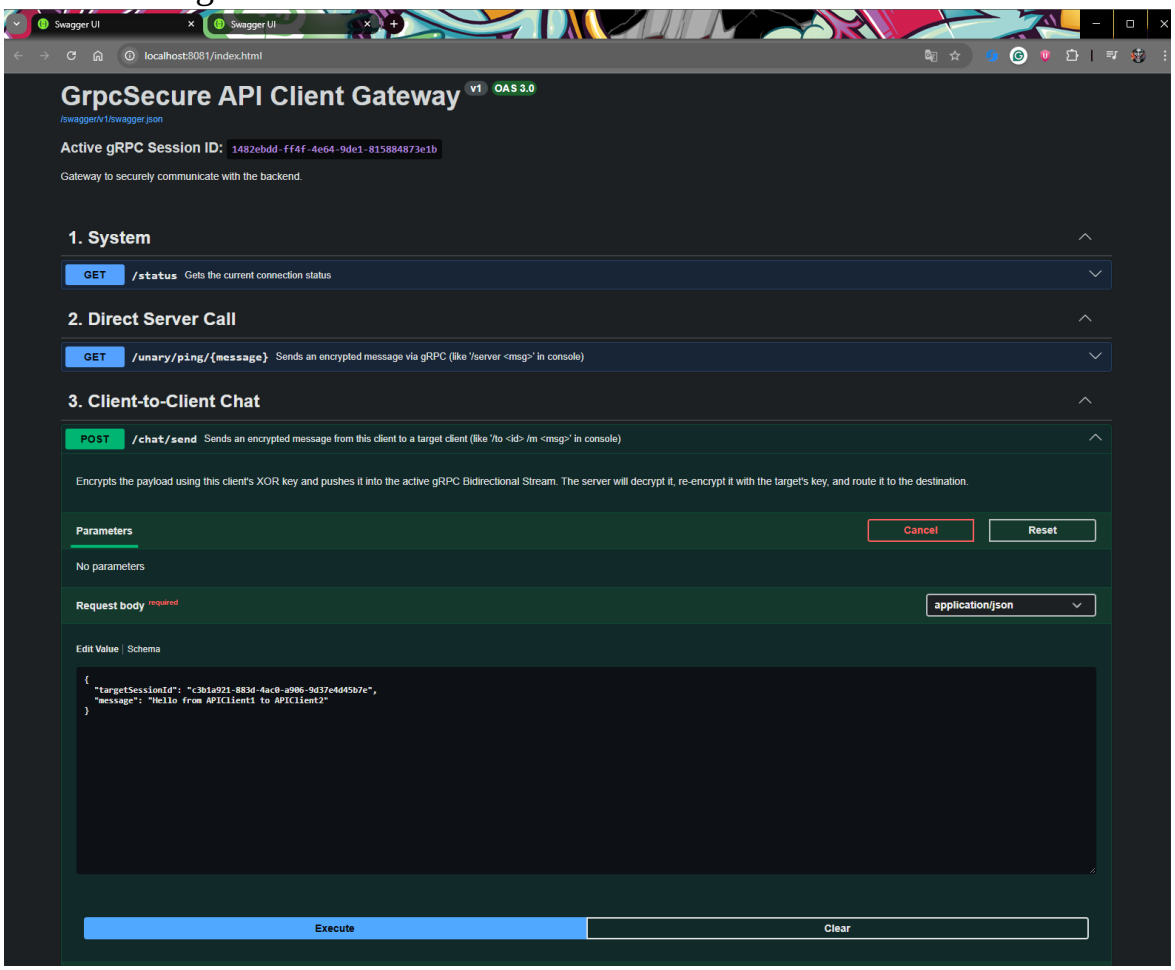
```
[System]: 4. Type 'exit' to quit.
> /to 1482ebdd-ff4f-4e64-9de1-815884873e1b /m Test from console
[System]: Recipient set to [1482ebdd-ff4f-4e64-9de1-815884873e1b].
> info: System.Net.Http.HttpClient.SecureCommunicationClient.ClientHandler[101]
      Received HTTP response headers after 1165292.2156ms - 200
info: System.Net.Http.HttpClient.SecureCommunicationClient.LogicalHandler[101]
      End processing HTTP request after 1165294.4518ms - 200
[1482ebdd-ff4f-4e64-9de1-815884873e1b] says: Test from APIClient1
>
```

Крок 4: Взаємодія між двома веб-шлюзами (APIClient1 -> APIClient2)

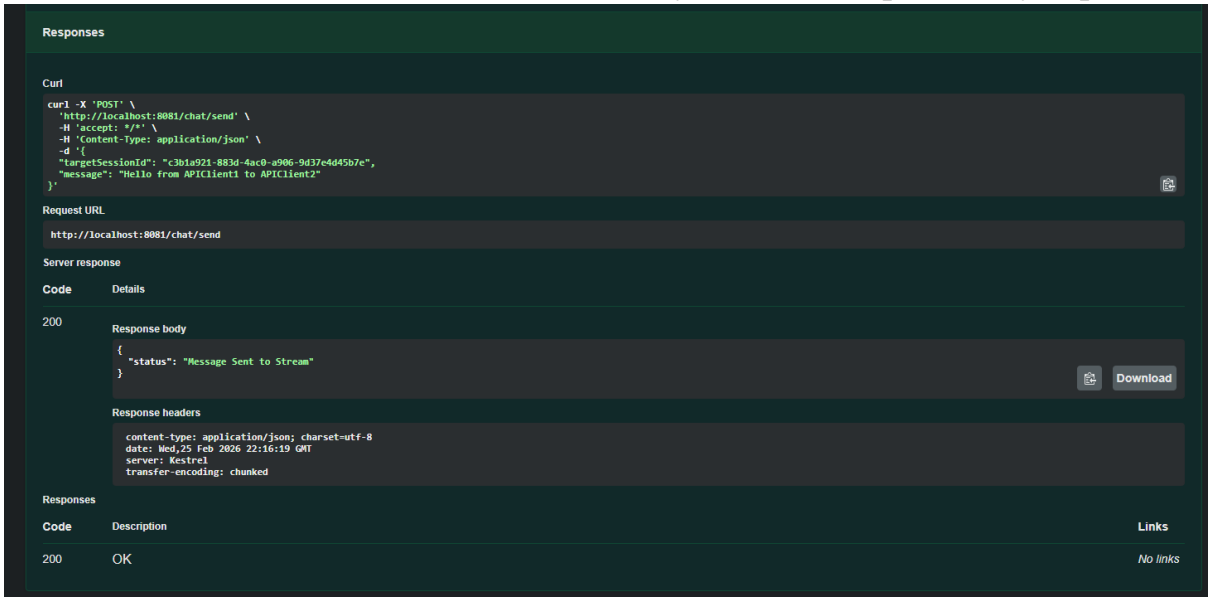
Тепер продемонструємо обмін даними між двома екземплярами API, щоб підтвердити незалежність сесій. В інтерфейсі API Client 1 знову використовуємо ендпоінт /chat/send. Цього разу ціллю вказуємо Session ID другого шлюзу (API Client 2):

TargetSessionId: c3b1a921-883d-4ac0-a906-9d37e4d45b7e (ID API Client 2)

Message: "Hello from APIClient1 to APIClient2"

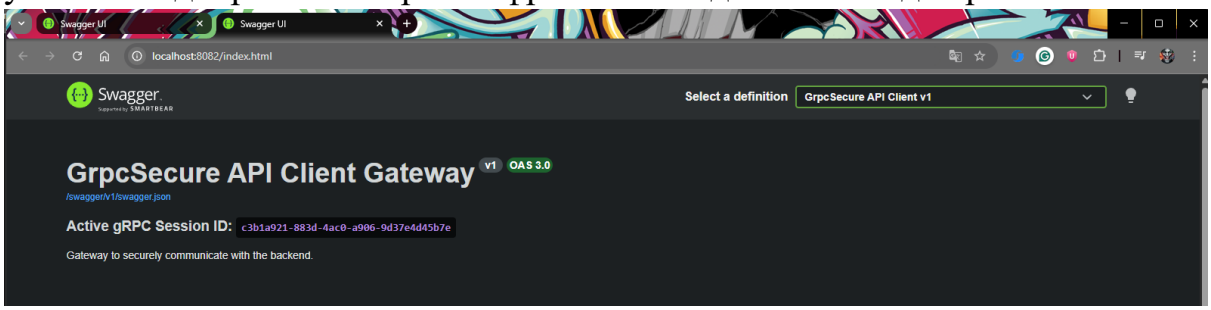


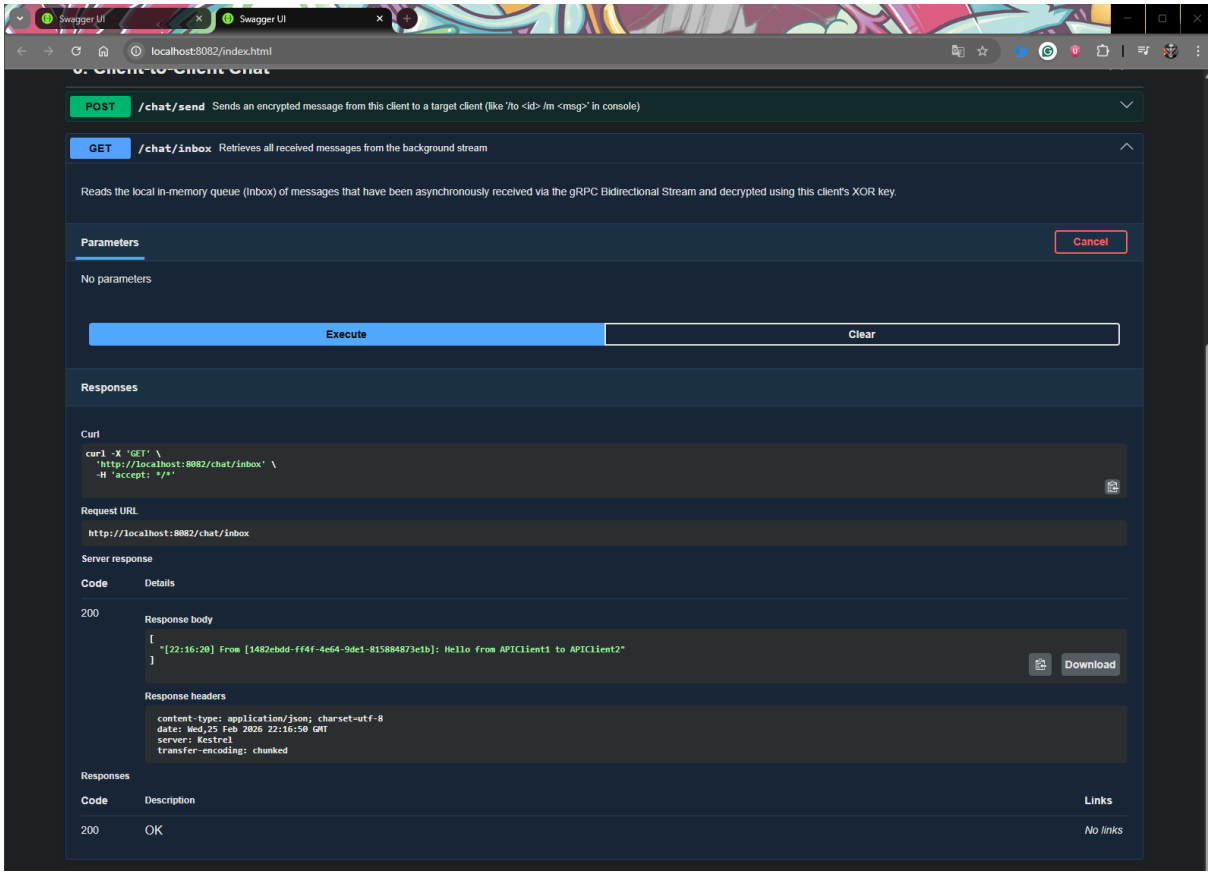
Натискаємо Execute. Повідомлення успішно відправлено у стрім.



Крок 5: Перевірка Inbox та зворотна відповідь (APIClient2 -> APIClient1)

Відкриваємо сторінку Swagger для API Client 2 (наприклад, http://localhost:8082). Виконуємо GET-запит до /chat/inbox. У тілі відповіді успішно відображається розшифроване повідомлення від першого клієнта:

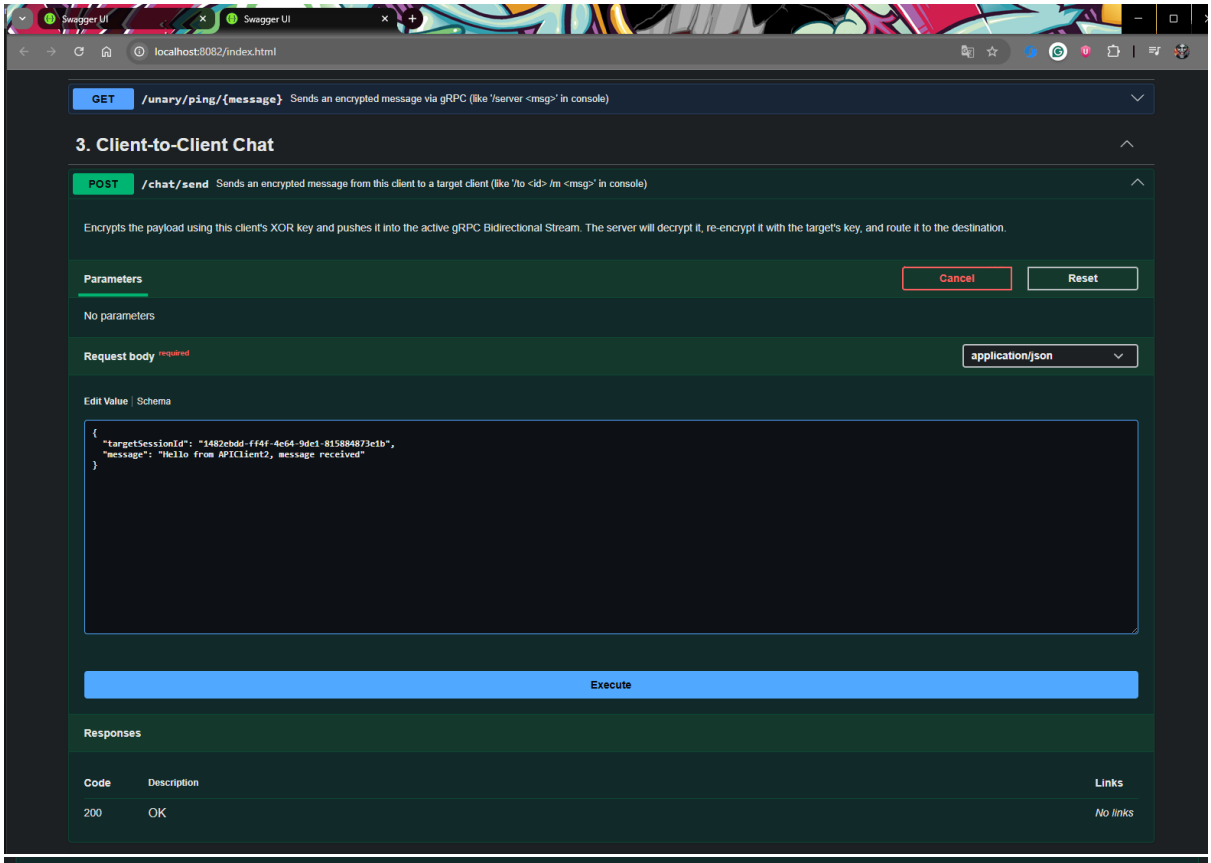


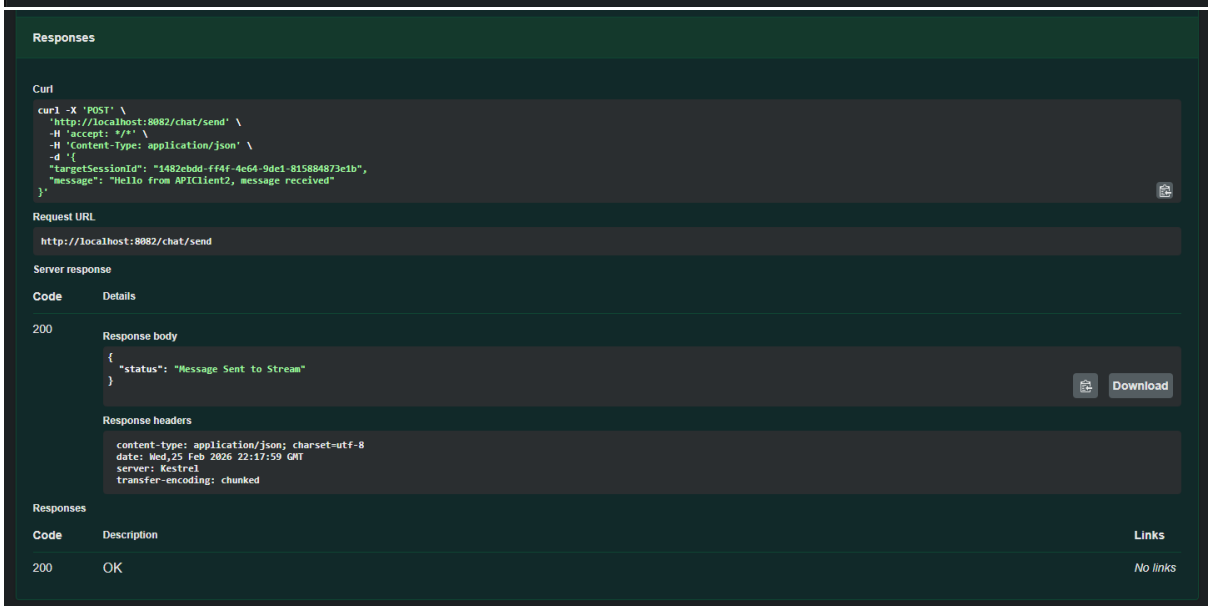


Щоб відповісти, у цьому ж вікні виконуємо POST-запит до /chat/send:

TargetSessionId: 1482ebdd-ff4f-4e64-9de1-815884873e1b (ID API Client 1)

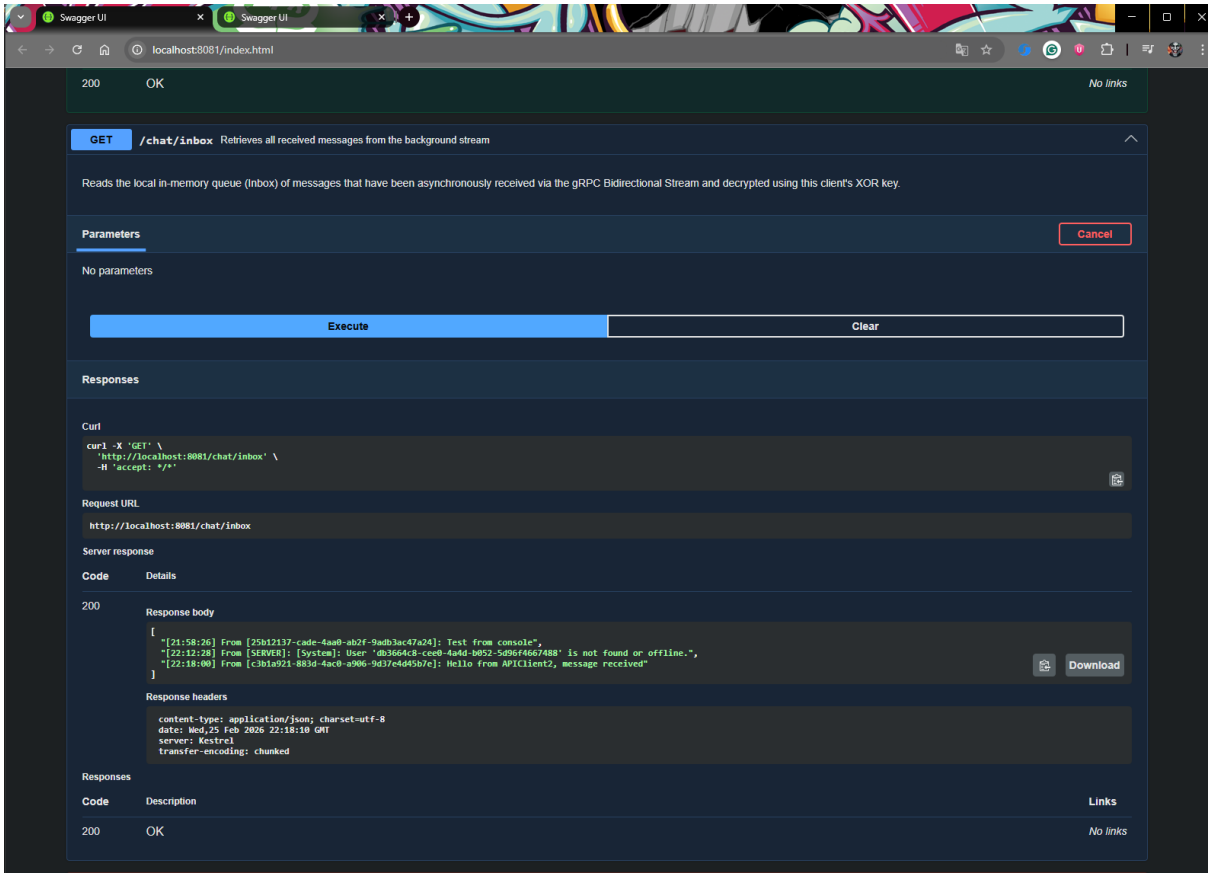
Message: "Hello from APIClient2, message received"





Крок 6: Фінальна перевірка отримання

Повертаємося на вкладку API Client 1 та ще раз викликаємо GET /chat/inbox. У масиві повідомлень бачимо нове повідомлення від API Client 2.



Система успішно маршрутизує та перешифровує повідомлення між будь-якими активними учасниками мережі у реальному часі.

Цей сценарій повністю підтверджує успішну реалізацію асинхронного, зашифрованого стрімінгу повідомлень між різнорідними клієнтами з використанням сервера як безпечного брокера.

Висновки: за результатом виконання лабораторної роботи спроєктовано та реалізовано захищену розподілену клієнт-серверну систему обміну повідомленнями на базі технології віддаленого виклику процедур (gRPC). Головною метою було створення надійного середовища для взаємодії незалежних клієнтів із забезпеченням конфіденційності даних. Цю мету було досягнуто завдяки інтеграції алгоритму симетричного шифрування (XOR) із механізмом динамічної генерації унікальних сесійних ключів під час початкової аутентифікації (Handshake). Додатково було успішно реалізовано централізовану маршрутизацію повідомлень через двонаправлений потік (Bidirectional Streaming) та забезпечено взаємодію між консольними додатками і веб-шлюзами (API Gateway) у контейнеризованому середовищі Docker.

Вихідний код застосунку можна знайти за наступним посиланням на GitHub.